FREE PLAYBOOK EDITION

The Agentic Developer

Stop Writing Code. Start Directing AI.

Five essential chapters pulled from a comprehensive guide to building software with AI agents. Learn the mindset shift, the discipline, and the workflows that separate productive agentic developers from those stuck in productivity illusions.

By Julio Casal

Software Engineer, Ex-Microsoft, Founder of .NET Academy

What You'll Learn

This playbook contains five chapters from the complete 29-chapter book, chosen for immediate impact and practical value.

Chapter 1

Software Development Has Changed

From autocomplete to autonomous agents. What actually changed, why it matters, and why "just use Copilot" isn't a strategy.

Chapter 2

The METR Paradox

Why experienced developers are 19% slower with AI tools yet believe they're 20% faster. The discipline gap that separates good from great.

Chapter 3

Your New Job Description

The five new skills you need to thrive as an agentic developer. How your coding expertise becomes even more valuable.

Chapter 4

Writing Effective Context Files

The single most important skill. How to curate what agents see so they produce better results. Complete templates included.

Chapter 5

The TDD Agent Loop

Write tests first, let the agent write the code. The highest-confidence workflow for working with AI, with complete examples.

Introduction

A Practical Guide to Building Software with AI Agents

By Julio Casal

About the Author

Julio Casal is a software engineer, ex-Microsoft, and founder of .NET Academy (dotnetacademy.io). He builds real systems, teaches thousands of developers, and has spent the last year going deep on agentic workflows so you don't have to learn the hard way.

What This Playbook Is

This is not a hype piece. This is not "AI will replace developers" clickbait.

This is a field guide. Five chapters pulled from the full 29-chapter book, chosen because they represent the most important ideas you need right now: what changed, why it matters, and how to actually do the work.

Here's the problem. Developers are adopting AI tools at a staggering rate. Anthropic's 2026 report says developers now use AI in 60% of their work. GitHub Copilot has generated over a million pull requests. Every IDE ships an "agent mode." The tools are everywhere.

But a landmark study from METR found that experienced open-source developers were actually 19% *slower* when using AI tools on familiar codebases. And here's the kicker: those same developers *believed* they were 20% faster.

That gap between perception and reality is where most developers live right now. They're using AI. They think it's helping. And in many cases, it's making them worse.

This playbook exists because the difference between those two outcomes, slower versus faster, isn't the tool. It's the technique. The developers who are genuinely more productive with AI agents aren't the ones who type the best prompts. They're the ones who provide the best context, write the best specs, and review the output with the sharpest eye.

They've become agentic developers.

What You'll Learn

Chapter 1: Software Development Has Changed sets the stage. From autocomplete to autonomous agents, the evolution happened fast. This chapter traces it and explains why "just use Copilot" isn't a strategy.

Chapter 2: The METR Paradox digs into the study that should make every developer uncomfortable. AI can make you slower. Understanding why is the first step to making sure it doesn't.

Chapter 3: Your New Job Description redefines what it means to be a developer in 2026. You're not a typist anymore. You're an architect, a reviewer, and an orchestrator.

Chapter 4: Writing Effective Context Files (Chapter 6 in the full book) is the most practical chapter here. Context engineering is the single most important skill in agentic development. This chapter gives you templates, examples, and a system.

Chapter 5: The TDD Agent Loop (Chapter 13 in the full book) is a complete workflow. Write the tests. Let the agent write the code. Run the tests. Iterate. It's the highest-confidence way to work with AI agents, and this chapter walks through it start to finish with real code.

A Note on Language

The code examples in this book use C# because that's what I write every day. But every pattern, every workflow, every principle applies to whatever language you use. If

you write Python, TypeScript, Go, Java, or Rust, this book is for you. The agent doesn't care about your language. It cares about your context.

Let's Go

You can read this playbook in an afternoon. By the end, you'll have a fundamentally different understanding of how to work with AI agents, and a concrete workflow you can start using on Monday.

The full book covers 29 chapters across seven parts, including parallel agent workflows, CI/CD integration, multi-agent coordination, .NET-specific patterns, security review, cost management, and a 30-day transformation plan. Details on what's in the full version are at the end of this playbook.

But first: let's talk about what changed.

About the Author

Julio Casal is a software engineer, ex-Microsoft, and founder of .NET Academy where he teaches thousands of developers. He's spent the last year studying agentic workflows so you don't have to learn the hard way. He builds real systems, teaches what works, and skips the hype.

This playbook is five chapters from a complete 29-chapter guide on agentic development. Details on the full book are at the end.

CHAPTER 1

Software Development Has Changed

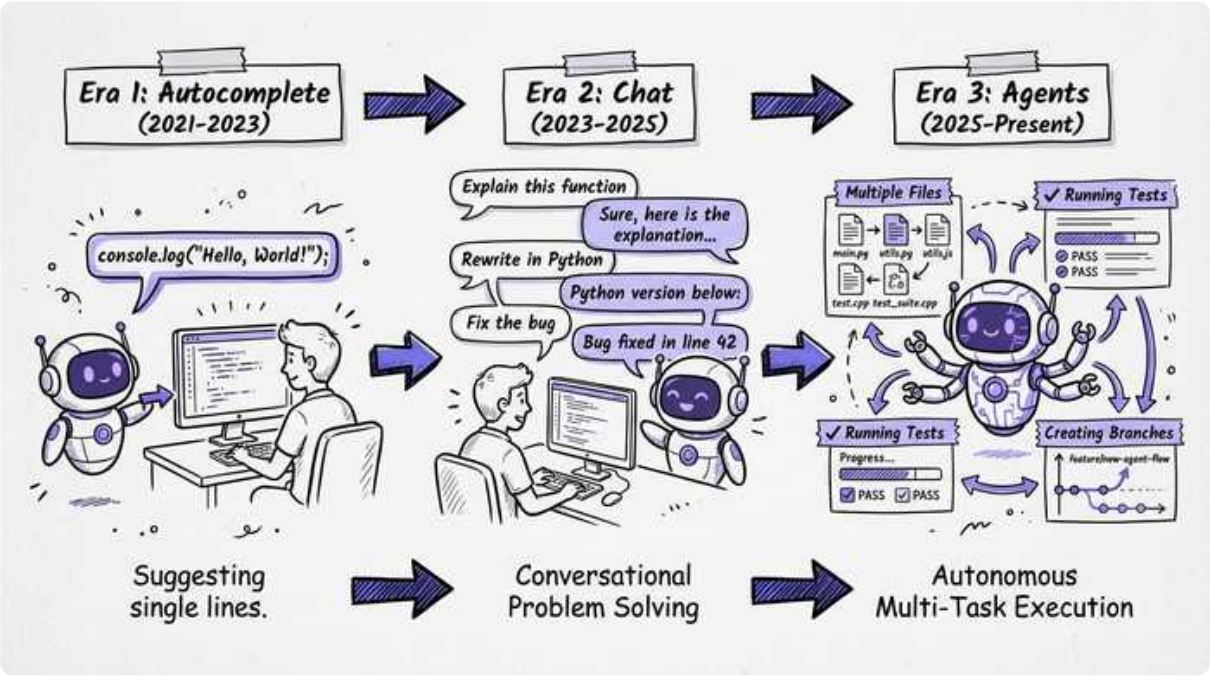
There's a moment most developers remember. Maybe it was the first time GitHub Copilot completed an entire function for you. Maybe it was the first time you described a feature in plain English and watched an agent write, test, and commit the code. Maybe it was the moment you realized you'd spent the last hour *reviewing* code instead of *writing* it.

Whatever the moment was, it split your career into before and after.

Software development has changed. Not in the way pundits predicted (no, developers are not obsolete), and not in the way skeptics hoped (no, this isn't just better autocomplete). It changed in a way that's both more subtle and more profound: the act of building software has shifted from writing code to directing the writing of code.

This chapter traces that shift. Not to rehash the history of AI for its own sake, but because understanding the trajectory tells you where to aim.

The Three Eras



The Three Eras Timeline

Era 1: Autocomplete (2021-2023)

GitHub Copilot launched in June 2021, and at first, it was exactly what it looked like: fancy autocomplete. You'd type a function signature, and it would suggest a body. You'd write a comment, and it would generate matching code. Useful? Absolutely. Transformative? Not yet.

The autocomplete era had a clear interaction model. You drove. The AI rode shotgun. It could suggest the next few lines, but it had no understanding of your project, your architecture, or your intent beyond the immediate file. It operated on a single-file context window, and that context window was small.

GitHub's own research during this period showed that developers using Copilot completed 26% more tasks per week. Meaningful, but incremental. You were still writing code. You just had a fast typist looking over your shoulder.

The mental model was simple: AI helps you type faster. Your job didn't change. Your skills didn't change. You just had a productivity boost on the mechanical parts of coding.

Era 2: Chat and Inline Assistance (2023-2025)

Then came ChatGPT, and with it, a different interaction model. Instead of autocomplete, you had a conversation. You could describe a problem and get a solution. You could paste an error message and get a diagnosis. You could ask "how does OAuth 2.0 work?" and get an explanation tailored to your codebase.

This era introduced chat panels in IDEs. Copilot Chat. Cursor's chat interface. Cody. The interaction model shifted from "complete my line" to "answer my question" and eventually to "edit this code for me."

The productivity gains were real but uneven. Developers who were already strong could use chat as a high-bandwidth rubber duck: a thinking partner that happened to know every framework and library. But developers who were struggling often got worse. They'd accept generated code they didn't understand, introducing bugs they couldn't debug.

This era also surfaced a critical insight that would define everything that followed: the quality of the output was directly proportional to the quality of the input. Vague questions got vague answers. Specific, well-structured prompts got excellent code. The skill wasn't "using AI." The skill was communicating precisely.

Era 3: Agents (2025-Present)

In 2025, the interaction model changed again, and this time the change was fundamental. The AI stopped being a tool you used and started being an agent you directed.

The difference matters. A tool does what you tell it, one step at a time. An agent takes a goal, breaks it into steps, executes them, observes the results, and adjusts. Agent mode in Copilot, Cursor, and Windsurf didn't just suggest code. It could:

- Read multiple files across your codebase
- Run your tests and see the output

- Execute terminal commands
- Create and delete files
- Iterate on its own work based on errors

Then came the CLI agents: Claude Code, Codex CLI. These operated directly in your terminal, with access to your full development environment. They could grep your codebase, run your build, execute your test suite, and iterate until things worked.

Then came background agents: GitHub Copilot Coding Agent, Codex Cloud, Google Jules. These didn't even need your IDE open. You could assign a task from your phone, and the agent would create a branch, write the code, run the tests, and open a pull request for your review.

Andrej Karpathy, who coined "vibe coding" in February 2025, evolved his terminology by February 2026 to "agentic engineering," calling it "a serious engineering discipline involving autonomous agents." The casual, throw-a-prompt-and-see-what-happens era was over.

What Actually Changed

The shift from autocomplete to agents isn't just a feature upgrade. It changed the fundamental economics of software development.

Writing Code Became Cheap

This is the hardest thing for experienced developers to internalize. For your entire career, the scarce resource was the ability to translate ideas into working code. That's what companies paid for. That's what made you valuable.

Writing code is no longer the bottleneck. An agent can generate hundreds of lines of working code in seconds. The GitHub/Microsoft study found that Copilot users completed 26% more tasks, effectively turning an 8-hour day into 10 hours of output. At scale, GitHub reported up to 81% productivity improvement among active Copilot users. Over a million pull requests were created by agents between May and September 2025 alone.

The raw production of code is cheap and getting cheaper.

Reading Code Became Expensive

Here's what nobody talks about enough: as code generation gets cheaper, code *review* gets more expensive. Not in dollars, but in cognitive load.

When an agent generates 200 lines of code in 30 seconds, someone has to verify that those 200 lines are correct, secure, performant, and maintainable. That someone is you. And reviewing code you didn't write is harder than reviewing code you did write, because you don't have the mental model of the decisions that were made along the way.

Addy Osmani, engineering lead on Google Chrome and author of *Beyond Vibe Coding*, put it precisely: "You're trading typing time for review time, implementation effort for orchestration skill, writing code for reading and evaluating code."

This is the new trade. And it's not a bad one. But it requires different skills than most developers have spent their careers building.

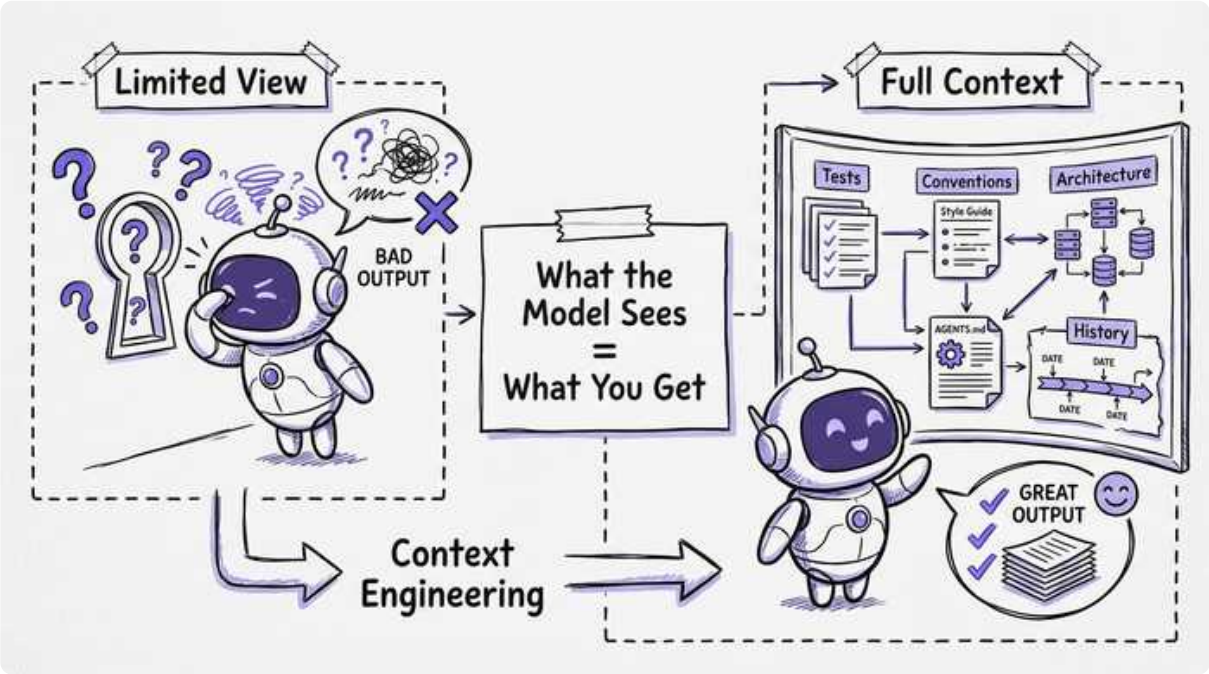
Specification Became the Product

In the agent era, the specification IS the work. A vague request like "build me a REST API for user management" will produce code. It will compile. It might even pass basic tests. But it will be riddled with assumptions you didn't intend, patterns you didn't want, and edge cases you didn't consider.

A precise specification, one that defines the endpoints, the data model, the error handling behavior, the authentication scheme, the test scenarios, and the architectural constraints, will produce code that's genuinely useful. Often better than what you'd have written by hand, because the agent can apply patterns consistently across an entire codebase without fatigue or distraction.

Simon Willison, one of the most prolific practitioners of agentic development, made this observation: "Code that started from your own specification is a lot less effort to review." That's the key insight. The spec isn't just input to the agent. It's input to your future self, the reviewer.

Context Became King



Context Engineering Concept

The single biggest differentiator between developers who thrive with agents and those who struggle is context engineering. Not prompting. Context engineering.

Prompting is what you type into the chat box. Context engineering is everything else: the files that tell the agent about your project, the rules that guide its behavior, the architectural decisions documented in markdown, the test suite that validates its output, and the codebase itself.

Bharani Subramaniam from ThoughtWorks defined it simply: "Context engineering is curating what the model sees so that you get a better result."

The developers who are genuinely faster with AI agents have invested in making their codebases legible to agents. They have context files that explain their conventions. They have comprehensive test suites that catch mistakes. They have clear architectural boundaries that prevent agents from making cross-cutting changes they shouldn't. They haven't just adopted tools. They've adapted their entire development environment.

The Uncomfortable Truth

Let's be honest about something. Most developers are in a weird middle ground right now. They're using AI tools. They have Copilot or Cursor or Claude Code installed. They use them daily. And many of them are getting worse, not better.

The METR study (which we'll dig into deeply in the next chapter) found that experienced developers were 19% slower with AI tools on familiar codebases. But they *perceived* themselves to be 20% faster. That's not a small discrepancy. That's a 39-percentage-point gap between perception and reality.

This happens because AI tools are seductive in a very specific way. They remove the *feeling* of effort. When an agent writes 200 lines of code for you, it feels like you accomplished something. When you then spend 45 minutes debugging a subtle issue the agent introduced, that feels like a separate problem, not an AI problem. The human brain doesn't naturally attribute the debugging time back to the generation that caused it.

The result is that most developers are in what I call the "productivity illusion": using AI tools, feeling productive, and actually being less effective than they would be without them.

This book exists to get you out of that illusion and into genuine, measurable productivity gains.

What the Best Developers Are Doing Differently

The developers who are genuinely thriving with agents, the ones who are 2-3x more productive, not 19% slower, share a set of practices. These practices are the subject of this entire book, but here's the preview:

They write specs before prompts. They don't start with "build me a thing." They start with a design document, a set of requirements, or at minimum a detailed description of what they want. The spec becomes the prompt.

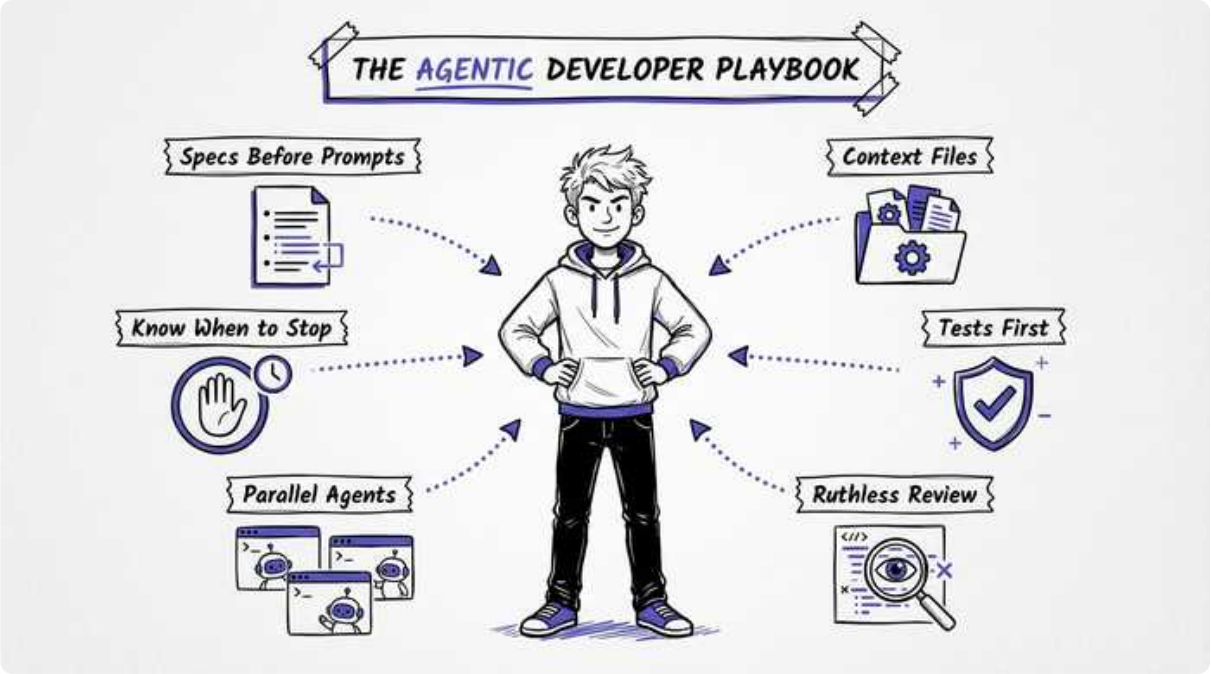
They invest in context files. Their projects have AGENTS.md or CLAUDE.md files that tell agents how the project works. Coding conventions, architecture decisions, build commands, testing frameworks. This context makes every agent interaction better.

They write tests first. This is the single highest-leverage practice. When you have a comprehensive test suite, you can let an agent iterate freely because the tests catch mistakes automatically. The agent writes code, runs the tests, fixes failures, and repeats. You review the final result, not every intermediate step.

They review ruthlessly. They don't accept generated code at face value. They read it with the same (or greater) scrutiny they'd apply to a pull request from a junior developer. They question architectural decisions. They check edge cases. They verify security implications.

They work in parallel. Following Simon Willison's "parallel coding agent lifestyle," they run multiple agents simultaneously on different tasks. One agent researches a library, another fixes a bug, a third implements a feature. The developer orchestrates and reviews.

They know when to stop. They recognize when an agent is going in circles, when the task is better done by hand, and when the cost of context-switching to agent mode exceeds the benefit. Not every task should be agentic.



The Agentic Developer Playbook

The Anthropic Forecast

Anthropic's 2026 Agentic Coding Trends Report identified eight trends reshaping software development. The first is the most relevant: a "tectonic shift in the SDLC" where engineers move from writing code to agent supervision, system design, and output review.

This isn't a future prediction. This is a description of what's already happening at companies that have adopted agentic workflows seriously. Engineers are spending less time in editors and more time in specifications and reviews. They're writing less code and reading more code. They're making fewer keystrokes and more decisions.

The report also notes that agents are becoming "team players," coordinating with each other in multi-agent systems. One agent handles the API layer, another handles the frontend, a third writes tests, and an orchestrator manages the coordination. This is no longer science fiction. It's available today in tools like Claude Code's sub-agent system and the ComposioHQ agent-orchestrator.

What This Means For You

If you're reading this, you're probably in one of three positions:

You're skeptical. You've tried AI tools, found them overhyped, and gone back to writing code the old way. The METR study probably validates your experience. And you're right: naive AI tool use *does* hurt. But disciplined agentic development is a different thing entirely. Give this playbook a chance. The workflows here are nothing like "ask ChatGPT to write your code."

You're enthusiastic but frustrated. You use AI tools every day but the results are inconsistent. Sometimes the agent nails it. Sometimes it produces garbage. You can't figure out the pattern. The pattern is context. Chapter 4 of this playbook will change how you think about it.

You're already in. You've adopted agentic workflows and you're looking to sharpen your technique. You want the specific patterns, the templates, the workflows that separate good from great. That's what the TDD Agent Loop chapter (and the full book) deliver.

Wherever you're starting from, here's the promise: by the end of this playbook, you'll understand the shift, you'll know why disciplined technique matters, and you'll have a concrete, battle-tested workflow you can start using immediately.

Software development has changed. The question isn't whether to adapt. It's how fast.

CHAPTER 2

The METR Paradox

In July 2025, a research organization called METR published a study that should have been a wake-up call for every developer using AI tools. It wasn't. Most people either ignored it, dismissed it, or cherry-picked the parts that confirmed what they already believed.

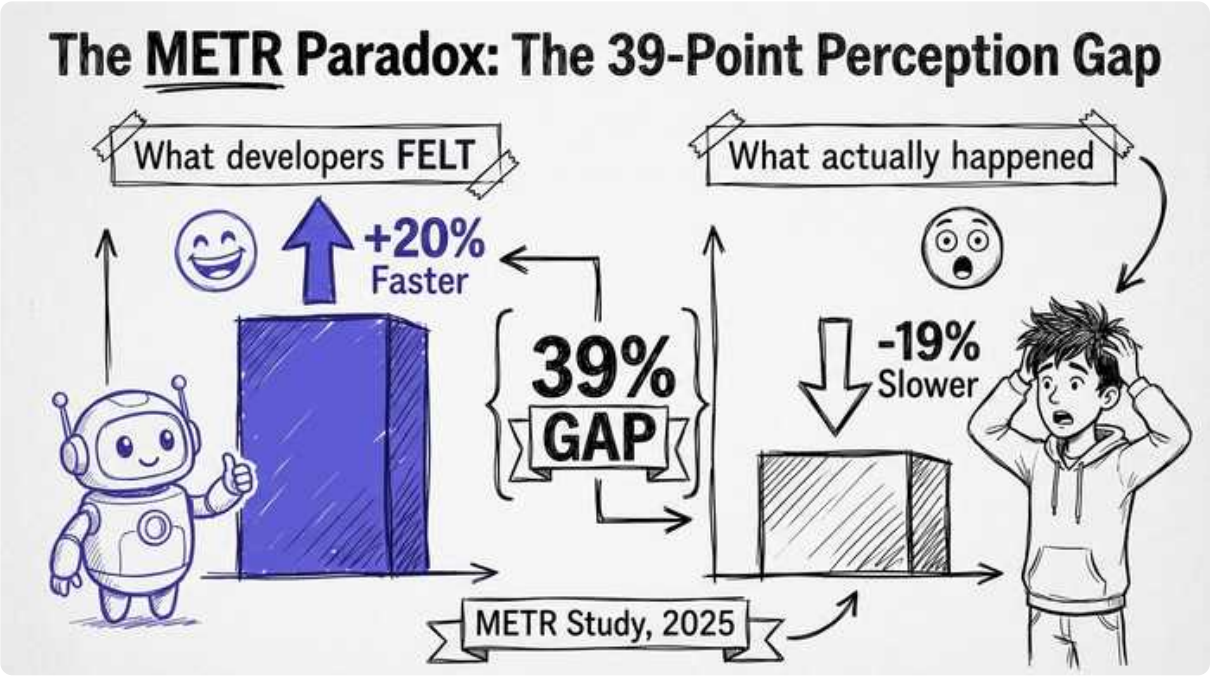
Here's what the study found: experienced open-source developers working on codebases they knew well were **19% slower** when using AI tools compared to working without them.

Let that land for a second. Not beginners. Experienced developers. Not unfamiliar codebases. Repositories they had contributed to, understood deeply, and navigated daily. The AI tools didn't help them. The tools actively slowed them down.

But here's the part that turns this from an interesting finding into something genuinely unsettling: those same developers **believed they were 20% faster**.

That's a 39-percentage-point gap between perception and reality. And it tells us something critical about the current state of AI-assisted development that no amount of marketing from tool vendors will address.

The Study



The 39-Point Perception Gap

The METR study (published as a preprint on arXiv) was rigorous in ways that most AI productivity claims are not. They didn't survey developers about how they felt. They measured actual task completion times. Experienced open-source contributors worked on real issues in repositories they maintained, with and without AI assistance, in a controlled setting.

The tasks were representative: bug fixes, feature implementations, refactoring. Real work, not coding challenges or toy problems. The kind of tasks that make up a developer's actual week.

And the result was clear. AI tools slowed these developers down.

Why AI Makes Experts Slower (When Used Naively)

The finding sounds counterintuitive until you think about what actually happens when an experienced developer uses an AI tool on familiar code.

The Context Switch Tax

When you know a codebase deeply, you have a rich mental model. You know where things are. You know why decisions were made. You know the hidden constraints that aren't documented anywhere. You can hold the relevant context in your head and write code that fits the system like a puzzle piece.

When you introduce an AI agent into this workflow, you're adding a context switch. Instead of writing code directly from your mental model, you now have to:

1. Translate your mental model into a prompt
2. Wait for the agent to generate code
3. Read the generated code
4. Compare it against your mental model
5. Identify discrepancies
6. Decide whether to fix the code or regenerate
7. Often, just rewrite it yourself anyway

For an expert working on familiar code, steps 1-6 take longer than just writing the code. The agent doesn't have your mental model. It doesn't know about the conversation you had with a teammate three months ago that explains why the authentication middleware is structured that way. It doesn't know that the seemingly redundant null check exists because of a specific edge case in production.

The Illusion of Progress

So why did these developers *think* they were faster? Because AI tools create a compelling illusion of productivity.

When an agent generates 50 lines of code in two seconds, your brain registers that as "I just did 50 lines of work." It feels fast because the *visible output* appeared fast. The invisible work, reading it, verifying it, fixing the three subtle issues, adjusting the approach when it doesn't quite fit, all gets mentally filed under "normal development work," not "overhead from using AI."

This is a well-documented cognitive bias. Humans are bad at attributing effort to the right cause, especially when the cause and the effort are separated in time. The generation feels instant. The debugging happens ten minutes later. Your brain doesn't connect them.

The Sunk Cost Trap

There's another dynamic at play. When an agent generates a large block of code, developers feel reluctant to throw it away. They've invested time in crafting the prompt, waiting for the response, and reading the output. Even when the code isn't quite right, there's a pull to "fix it up" rather than start fresh.

This is almost always the wrong choice. Fixing AI-generated code that doesn't match your mental model takes longer than writing correct code from scratch. But the sunk cost makes it feel wasteful to discard it.

The Other Side of the Data

Now, the METR study has important boundary conditions that matter enormously.

The study measured experienced developers on *familiar* codebases. This is a very specific scenario. And it's not the only scenario that matters.

Other studies paint a different picture for different contexts:

GitHub's research found that developers using Copilot completed 26% more tasks. The key difference: this included unfamiliar tasks and greenfield work, exactly the contexts where you *don't* have a deep mental model to draw on.

Individual developer output studies show 20-40% productivity increases when AI tools are used across a mix of familiar and unfamiliar work.

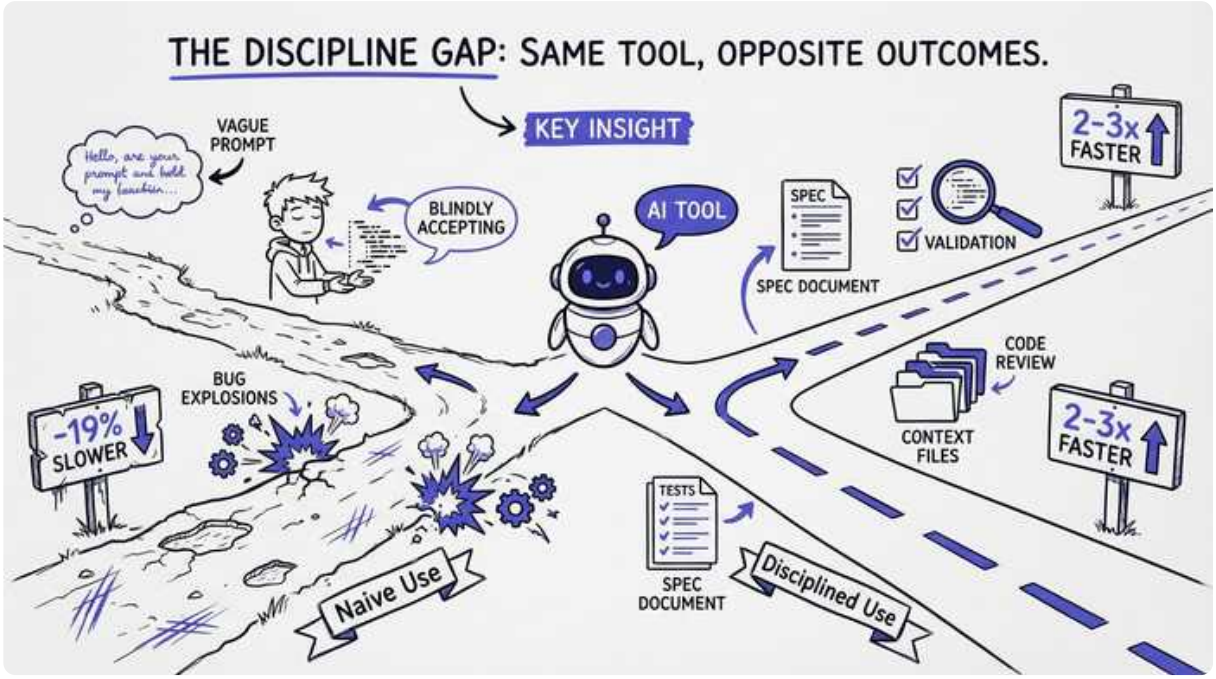
GitHub's scale data showed up to 81% productivity improvement among active Copilot users, measured across over a million pull requests created between May and September 2025.

So the full picture isn't "AI tools don't work." It's more nuanced and more useful:

- **AI tools hurt experts on familiar code** (when used naively)
- **AI tools help everyone on unfamiliar code** and greenfield work
- **AI tools help most on boilerplate, repetitive, and mechanical tasks**
- **The gap between "helps" and "hurts" is technique**

That last point is the one that matters for this book.

The Discipline Gap



The Discipline Gap

Here's the paradox, stated cleanly: the same tools that make naive users slower make disciplined users dramatically faster. The variable isn't the tool. It's the technique.

What does disciplined use look like? It looks like the practices we'll cover throughout this book:

1. Knowing When to Use Agents (And When Not To)

The developers who are genuinely more productive have developed an intuition for when an agent will help and when it will hurt. Rules of thumb:

Use an agent when:

- You're working in an unfamiliar part of the codebase
- The task is boilerplate-heavy (CRUD endpoints, data models, configuration)
- You need to implement a well-known pattern (repository pattern, middleware, API versioning)
- You're exploring a new library or framework
- The task is well-specified and test-coverable

Don't use an agent when:

- You can write the code faster than you can describe it
- The task requires deep knowledge of hidden constraints
- You're in a flow state and the code is flowing naturally
- The change is tiny (a one-line fix where you already know the line)

The METR developers didn't make this distinction. They used AI tools across the board, including on tasks where their expertise was the fastest path.

2. Investing in Context

The METR study participants used AI tools with minimal context engineering. They didn't have AGENTS.md files explaining their project conventions. They didn't have comprehensive test suites that could validate agent output automatically. They didn't provide architectural constraints or coding standards.

They did what most developers do: opened the agent, typed a prompt, and hoped for the best.

Disciplined agentic development looks nothing like that. It looks like a carefully engineered context stack (which we cover in Chapter 4) that gives the agent enough information to produce useful output on the first try.

3. Writing Tests First

This is the single biggest differentiator. When you have tests, you can let an agent iterate autonomously. It writes code, runs the tests, sees failures, and fixes them. The feedback loop is tight and automatic.

Without tests, you're the feedback loop. You read every line, evaluate every decision, and catch every bug manually. That's slow. That's what the METR developers were doing.

The TDD Agent Loop (Chapter 5 in this playbook) is designed specifically to address this. It's the workflow that turns the METR paradox on its head.

4. Reviewing Like an Architect

Disciplined developers don't just check if the code "works." They review agent output the way a senior architect reviews a pull request:

- Does this fit the existing patterns in the codebase?
- Are there security implications?
- How will this perform at scale?
- Is this maintainable by a human who hasn't seen the prompt?
- What's the test coverage?

This kind of review is a skill. It's learnable. And it's far more valuable than the ability to type code fast.

The GitClear Warning

While we're looking at data, one more study deserves attention. GitClear's 2025 analysis found a **4x growth in code clones** in repositories using AI assistants. That is, copied or near-identical code blocks appearing throughout codebases.

This is the quality side of the equation. Even when AI tools make you *faster* (in terms of task completion), they can make your codebase *worse*. Agents don't refactor. They don't say "this pattern already exists in your utility library, let me reuse it." They generate fresh code for every request, and that code often duplicates existing functionality.

Disciplined developers catch this during review. They look for duplication, unnecessary abstractions, and code that should have been a function call instead of a reimplementation. Undisciplined developers accept the output and move on, accumulating technical debt at 4x the normal rate.

The Real Lesson

The METR study isn't an argument against AI tools. It's an argument against naive AI tool use. And the distinction matters enormously.

Naive use: open the agent, type a prompt, accept the output, move on.

Disciplined use: engineer the context, write the tests, craft the specification, let the agent work within constraints, review the output thoroughly, iterate when needed, and know when to do the work by hand.

The gap between these two approaches is the gap between being 19% slower and being 2-3x faster. It's the gap between thinking you're productive and actually being productive.

This is why Addy Osmani observed that "AI-assisted development actually rewards good engineering practices MORE than traditional coding does." The better your specs, the better the AI's output. The more comprehensive your tests, the more

confidently you can delegate. The cleaner your architecture, the less likely the agent is to introduce inconsistencies.

Good engineering practices were always valuable. In the agent era, they're *essential*.

Your Calibration Check

Before moving on, try this exercise. Think about the last week of your development work. For each task where you used an AI tool, ask:

1. Was I actually faster, or did it just *feel* faster?
2. How much time did I spend debugging or fixing AI-generated code?
3. Did I accept any code I didn't fully understand?
4. Would I have been faster writing it by hand?

Be honest. The METR developers weren't, and the 39-percentage-point gap between their perception and reality is the proof.

If you find that AI tools are genuinely saving you time, great. You're probably already practicing some of the discipline this book teaches. Sharpen it further.

If you find that the tools might be creating a productivity illusion, that's not a failure. It's awareness. And awareness is the first step to the transformation the rest of this book provides.

The paradox is real. But it's solvable. That's what the next chapters are for.

CHAPTER 3

Your New Job Description

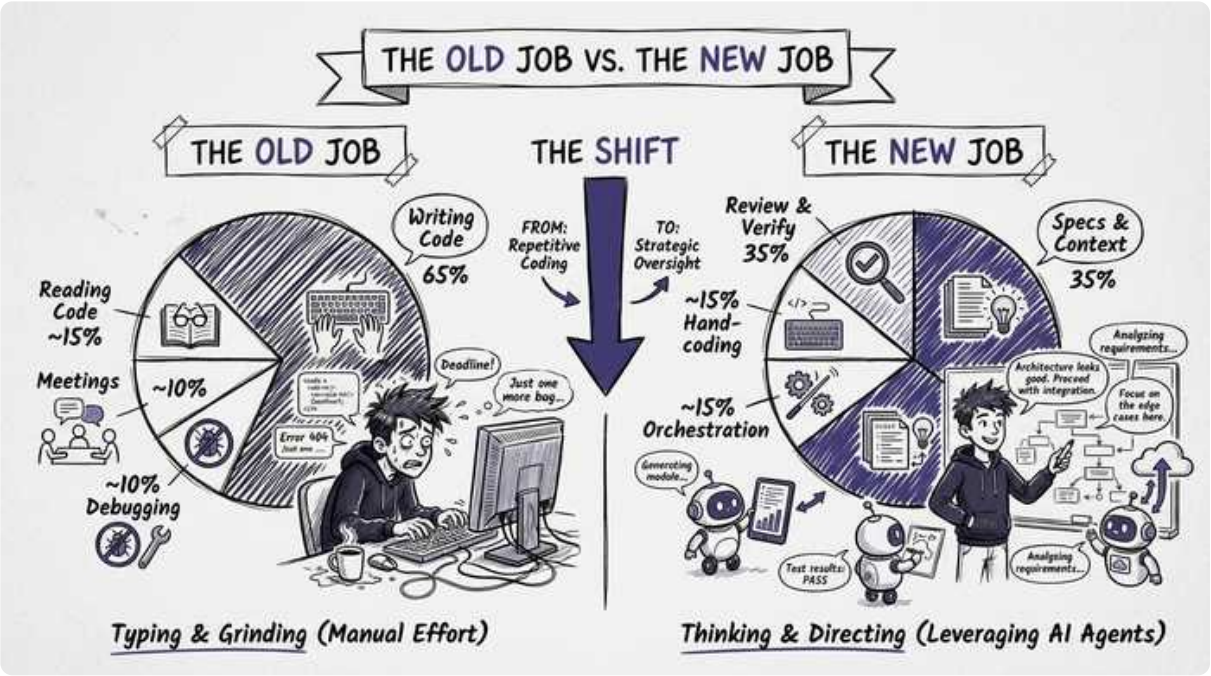
If you've internalized the first two chapters, you're sitting with an uncomfortable realization: the thing you've spent years getting good at (writing code) is no longer the primary thing your job demands.

That doesn't mean your skills are useless. It means they've been *promoted*. Everything you know about software architecture, system design, debugging, testing, and code quality is more valuable than ever. You just apply it differently now.

Addy Osmani captured this precisely: "Agentic engineering disproportionately benefits senior engineers." The more you know about building software, the better you are at directing agents to build it. Your experience isn't obsolete. It's your competitive advantage.

But the day-to-day work looks different. Let's define what the job actually is now.

The Old Job vs. The New Job



The Old Job vs. The New Job

What You Used to Do

A typical day for a senior developer in 2023 looked something like this:

- **Morning:** Read through open PRs, leave comments, merge what's ready
- **Mid-morning:** Pick up a ticket, read the requirements, start coding
- **Afternoon:** Write implementation code, debug issues, write tests
- **Late afternoon:** More implementation, maybe a design discussion
- **End of day:** Push code, update the ticket, plan tomorrow

The core activity: translating requirements into code. You spent maybe 60-70% of your time writing or editing code, 20% reading code, and 10% on everything else.

What You Do Now

A typical day for an effective agentic developer in 2026:

- **Morning:** Review PRs from overnight background agents, approve or send back with notes
- **Mid-morning:** Write specs for the day's work, break features into agent-sized tasks
- **Late morning:** Launch agents on 3-4 tasks in parallel, review first results
- **Afternoon:** Architecture decisions on edge cases agents flagged, review more output, refine context files based on patterns you see
- **Late afternoon:** Final reviews, integration testing, context file updates for tomorrow
- **End of day:** Queue up background agent tasks for overnight work

The core activity: specification, review, and orchestration. You spend maybe 20% of your time writing code by hand, 40% reviewing agent output, and 40% on specs, architecture, and context engineering.

That's not a small shift. That's a fundamentally different job.

The Five New Skills

The transition from code-writer to agentic developer requires five skills that most developers haven't deliberately practiced. They're all learnable. Some of them you already have in embryonic form.

1. Context Engineering

This is the big one. Chapter 4 of this playbook goes deep on it, so here's the short version: context engineering is the skill of curating what the agent sees so that it produces better results.

It includes:

- Writing and maintaining AGENTS.md or CLAUDE.md files

- Creating path-scoped rules for different parts of your codebase
- Designing your codebase to be AI-readable (clear naming, good structure, documented decisions)
- Knowing what context to include and, crucially, what to exclude
- Building and maintaining a test suite that serves as automated verification

This skill didn't exist two years ago. It's now the most important skill in your toolkit. The developers who are 2-3x more productive with agents are, almost without exception, excellent context engineers.

2. Specification Writing

Remember when "writing good requirements" was something PMs did and developers complained about? Now it's your core deliverable.

When you direct an agent, your specification IS the work. The more precise your spec, the better the output. The more ambiguous your spec, the more time you spend on review and rework.

Good specs for agents look like this:

```
## Task: Add rate limiting to the API

### Requirements
- Apply rate limiting to all endpoints under /api/v1/
- Use a sliding window algorithm with a 60-second window
- Default limit: 100 requests per window per API key
- Return 429 Too Many Requests with a Retry-After header when exceeded
- Rate limit state stored in Redis (use the existing IDistributedCache)
- Bypass rate limiting for requests with the X-Internal-Service header

### Constraints
- Do NOT modify the existing middleware pipeline ordering
- Use the existing ApiKey from the request context
(HttpContext.Items["ApiKey"])
- Follow the existing middleware pattern in src/Api/Middleware/

### Test Scenarios
- Request within limit returns 200
- Request exceeding limit returns 429 with correct Retry-After value
- Rate limit resets after window expires
- Internal service header bypasses rate limit
- Different API keys have independent limits
```

Compare that to: "Add rate limiting to the API." Same task. Vastly different output quality.

The spec isn't just for the agent. As Simon Willison observed, "Code that started from your own specification is a lot less effort to review." When you wrote the spec, you already know what the code *should* do. Reviewing becomes verification, not exploration.

3. Architectural Thinking

When agents write most of the code, the decisions that remain with you are the ones that matter most: the architectural decisions.

Which patterns to use. How to structure the project. Where to draw service boundaries. How data flows through the system. What trade-offs to make between complexity and flexibility. These are the decisions agents can't make well because they require understanding the full business context, the team's capabilities, the deployment constraints, and the long-term evolution of the system.

This is why Osmani says agentic engineering benefits senior engineers disproportionately. A junior developer can write a prompt. A senior developer can write a prompt *and* evaluate whether the resulting architecture will hold up under load, scale with the team, and survive the next three requirements changes.

If you've been coding for years and feel like your "just typing code" skills are becoming less valuable, here's the reframe: your architectural judgment is becoming *more* valuable. It's the thing agents can't replicate.

4. Code Review (At Scale)

You've done code review before. But you probably haven't done it at the volume and velocity that agentic development demands.

When you're running 3-5 agents in parallel (which is the workflow Simon Willison describes and we cover in the full book), you might review 10-20 substantial code changes in a single day. That's more review than most developers do in a week.

This requires a different approach than the leisurely PR review you're used to. You need:

A triage system. Not every agent output needs the same level of scrutiny. A boilerplate CRUD endpoint needs a quick scan. A security-sensitive authentication change needs line-by-line review. Develop a sense for what's high-risk and what's routine.

Pattern recognition. After reviewing hundreds of agent outputs, you'll start recognizing common failure modes: unnecessary abstractions, duplicated logic, missing edge cases, overly complex solutions to simple problems. Your review speed increases as your pattern library grows.

Automated guardrails. You shouldn't be the only line of defense. Comprehensive test suites, linting rules, static analysis, and CI pipelines catch the mechanical issues. Your

review focuses on the things machines can't catch: architectural fit, business logic correctness, and long-term maintainability.

The "would I approve this PR from a junior developer?" test. If the answer is "I'd send it back with comments," don't accept it from the agent either. Your quality bar doesn't change just because the code was free to produce.

5. Agent Orchestration

This is the meta-skill: knowing how to break work into pieces that agents can handle effectively, when to run agents in parallel versus sequentially, when to intervene versus let the agent iterate, and when to do the work yourself.

Good orchestration looks like:

- **Decomposing a feature** into 4-5 independent tasks that agents can work on simultaneously
- **Ordering dependent work** so that agent A's output feeds into agent B's context
- **Recognizing stuck agents** early and redirecting instead of letting them burn tokens going in circles
- **Switching tools** based on the task (CLI agent for terminal work, IDE agent for multi-file edits, background agent for overnight tasks)

Bad orchestration looks like:

- Giving an agent a vague, multi-day task and hoping for the best
- Running agents sequentially when they could work in parallel
- Spending 30 minutes crafting a prompt for a task you could code in 10 minutes
- Accepting agent output without review because "it generated fast, it must be right"

The Emotional Shift

Let's talk about something the technical literature ignores: this transition can feel terrible.

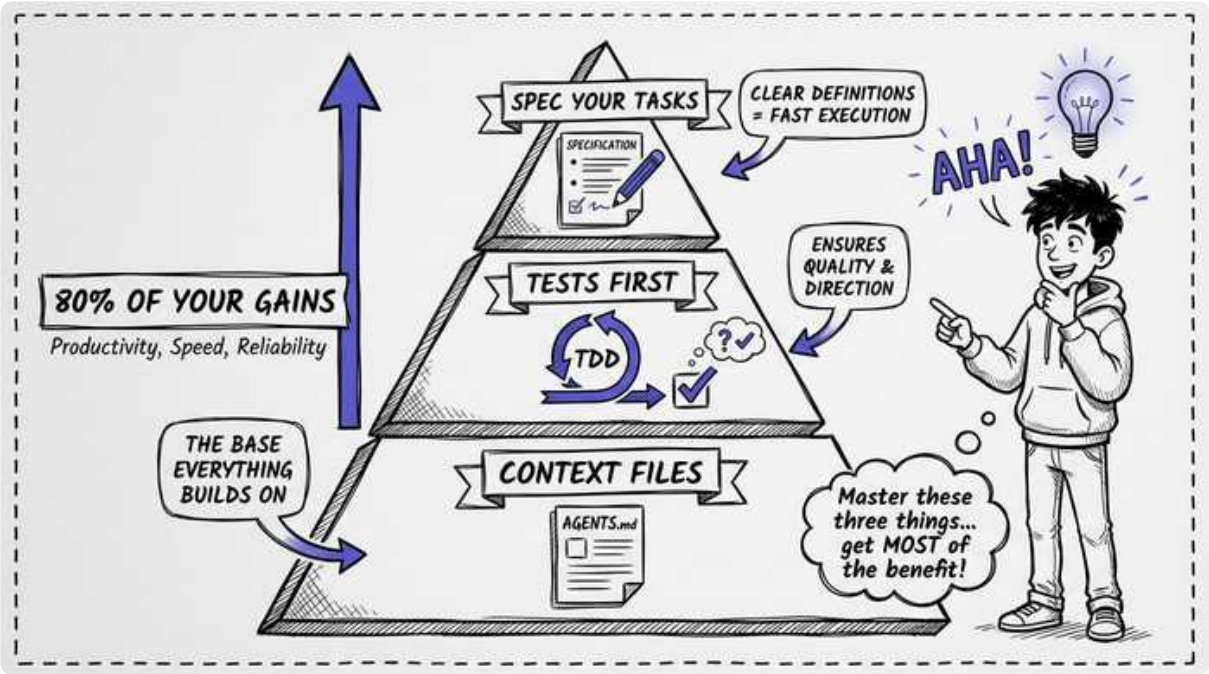
If you've spent 10+ years becoming an excellent code-writer, it can feel like the ground shifted under you. The thing you're great at is still useful, but it's no longer the main thing. That's disorienting. It can feel like a demotion even though it's objectively a promotion.

Here's the reframe that helped me: you're not losing the coding skill. You're adding skills on top of it. An architect who can also code is more valuable than an architect who can't. A reviewer who understands implementation deeply catches things a non-coding reviewer misses. Your coding skill makes you a *better* agentic developer, not an obsolete one.

The developers who struggle most with this transition are the ones who define their identity by their typing speed, their ability to hold an entire system in their head, or their pride in "doing it myself." These are all legitimate sources of professional pride. They're just no longer sufficient on their own.

The developers who thrive are the ones who redefine their identity around *outcomes*: shipping working software, making good architectural decisions, maintaining high code quality, and enabling their team (human and AI) to do their best work.

The 80/20 of Agentic Development



The 80/20 Pyramid

If you do nothing else after reading this playbook, do these three things. They account for roughly 80% of the productivity gains:

1. Create a Context File for Your Project

Even a simple AGENTS.md with your build commands, testing framework, coding conventions, and architectural decisions will dramatically improve agent output. Chapter 4 shows you exactly how. This takes 30 minutes and pays off on every single agent interaction going forward.

2. Write Tests Before Prompting

When you have tests, agents can self-correct. Without tests, you're the test suite. Chapter 5 walks through the TDD Agent Loop in detail. This single workflow change is responsible for the majority of productivity gains in my experience.

3. Spec Your Tasks

Before you prompt an agent, spend 5 minutes writing a clear specification. What should this code do? What are the constraints? What are the test scenarios? This small upfront investment saves 30+ minutes of review and rework on the other end.

These three practices, context files, tests first, and specs, are the foundation. Everything else in this book builds on them.

The 30-Day Horizon

Anthropic's report says developers now use AI in 60% of their work. That number is climbing. Within a year, developers who haven't adopted agentic workflows will be the exception, not the norm.

But here's the good news: this isn't a years-long transformation. The full book includes a 30-day plan (Chapter 29) that takes you from wherever you are today to a fully operational agentic workflow. The core skills, context engineering, specification writing, and the TDD agent loop, can be learned and practiced in weeks, not months.

You don't need to change everything at once. Start with the three practices above. Add context engineering depth as you go. Experiment with parallel agents when you're comfortable with single-agent workflows. Build toward the full agentic development lifecycle at your own pace.

The job has changed. But the new job is learnable, valuable, and honestly, more interesting than the old one. You make more decisions, solve harder problems, and ship more software. You just do less typing.

That's a trade worth making.

CHAPTER 4

Writing Effective Context Files

(Chapter 6 in the full book)

This is Chapter 6 in the full book.

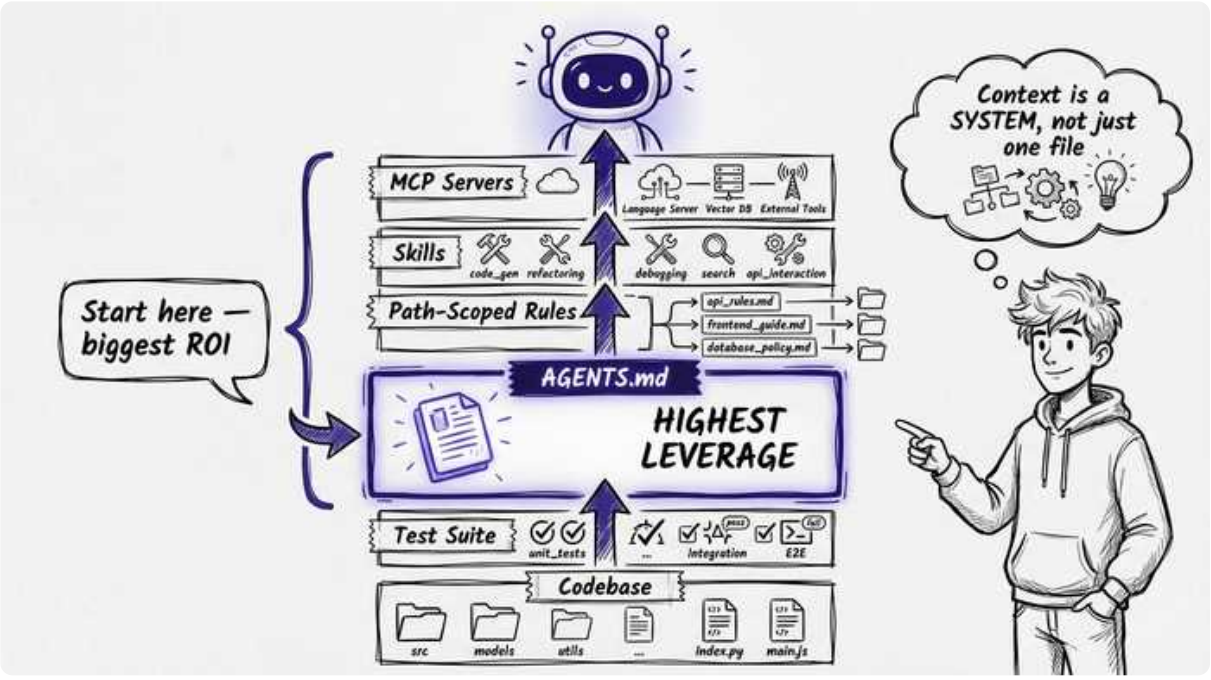
If you take one thing from this entire playbook, make it this: **context engineering is the single most important skill in agentic development.** Not prompting. Not tool selection. Not model choice. Context.

Bharani Subramaniam from ThoughtWorks defined it simply: "Context engineering is curating what the model sees so that you get a better result." That sounds abstract. In practice, it's concrete and mechanical. It's the set of files, rules, and conventions that tell your agent how to work with your specific project.

The developers who are 2-3x more productive with agents all have one thing in common: they've invested in context. They didn't just install the tool and start prompting. They built an information layer that makes every agent interaction better.

This chapter gives you the system. Templates, examples, and the reasoning behind every decision.

The Context Stack



The Context Stack

Modern AI coding agents load context from multiple layers. Understanding these layers tells you where to invest your effort.

Layer	What It Does	When It Loads	Your Time Investment
AGENTS.md / CLAUDE.md	Always-on project guidance	Automatically, every session	High (create once, maintain ongoing)
Path-scoped rules	Modular guidance for specific file types	When matching files are open	Medium (add as patterns emerge)
Skills	On-demand instructions and scripts	When the agent decides they're relevant	Low to medium
MCP servers	External tool and data access	When the agent needs external info	Setup cost, then low
The codebase itself	Your existing code IS context	Always	Ongoing (it's your normal work)
Test suite	Automated verification of agent output	When agent runs tests	High (but you should have this anyway)

The highest-leverage investment is the first row: your main context file. Everything else builds on it.

The Main Context File: AGENTS.md

There's an emerging standard called AGENTS.md (documented at agents.md). GitHub Copilot uses `copilot-instructions.md`. Cursor uses `.cursorrules`. Claude Code uses `CLAUDE.md`. The format varies but the concept is identical: a markdown file at the root of your project that tells agents how to work here.

I recommend using AGENTS.md because it's the most tool-agnostic, but the content matters more than the filename. Write it once and symlink or copy to tool-specific filenames if you need to.

What Goes In It

Your context file answers one question: "What does an agent need to know to write good code in this project?" Not everything about the project. Just the things that affect code quality.

Here's the structure I use, section by section.

Template: The Complete AGENTS.md

```
# AGENTS.md

## Project Overview
Brief description of what this project does. 2-3 sentences max.
Not a README. Just enough for the agent to understand scope.

## Tech Stack
- Runtime: .NET 9 / ASP.NET Core
- Database: PostgreSQL via EF Core
- Cache: Redis via IDistributedCache
- Message queue: RabbitMQ via MassTransit
- Auth: JWT bearer tokens with ASP.NET Identity
- Testing: xUnit + FluentAssertions + NSubstitute
- CI: GitHub Actions

## Build & Run
```

dotnet build

dotnet test

docker compose up -d

dotnet run --project src/Api

```
## Project Structure
```

src/

Api/ # ASP.NET Core Web API (entry point)

Domain/ # Domain entities and interfaces

Infrastructure/ # EF Core, external services, repositories

tests/

Api.Tests/ # Integration tests

Domain.Tests/ # Unit tests

```
## Coding Conventions
- Use file-scoped namespaces
- Use primary constructors for DI
- Records for DTOs, classes for entities
- No regions. Ever.
- Keep methods under 30 lines. Extract if longer.
- Async all the way down. No .Result or .Wait().

## Architecture Patterns
- Vertical slice architecture: each feature is a self-contained folder
- CQRS with MediatR: commands and queries as separate types
- Repository pattern for data access (no direct DbContext in handlers)
- Result pattern for error handling (no exceptions for business logic)

## Error Handling
- Return Result<T> from handlers, not exceptions
- Map Result failures to HTTP status codes in the controller
- Use ProblemDetails for all error responses
- Log errors in middleware, not in handlers

## Testing Conventions
- One test class per feature/handler
- Use the builder pattern for test data (see tests/Builders/)
- Integration tests use WebApplicationFactory with a real database
- Name tests: MethodName_Scenario_ExpectedResult

## What NOT to Do
- Do NOT use AutoMapper (we use manual mapping methods)
- Do NOT add NuGet packages without asking
- Do NOT modify the middleware pipeline in Program.cs without asking
- Do NOT use static methods for business logic
- Do NOT write console output (use ILogger)

## API Conventions
- RESTful routes: /api/v1/{resource}
```

```
- Use [FromRoute], [FromQuery], [FromBody] explicitly
- Return ActionResult<T> from controllers
- POST returns 201 with Location header
- All endpoints require [Authorize] unless explicitly noted
```

That's a real context file. It's opinionated, specific, and actionable. An agent reading this file knows how to write code that fits this project. Without it, the agent guesses, and its guesses are generic.

Why Each Section Matters

Project Overview: Agents make better decisions when they understand what they're building. A payments API and a chat application have different quality concerns.

Tech Stack: Prevents the agent from suggesting libraries you don't use. Without this, agents will happily suggest Dapper when you use EF Core, or Moq when you use NSubstitute.

Build & Run: Agents (especially CLI agents like Claude Code) can execute these commands. If they can run your tests, they can self-correct. This is critical for the TDD loop.

Project Structure: Agents need to know where to put new files. Without this, they'll create files in random locations or follow whatever convention the training data suggests.

Coding Conventions: This is where your context file pays the highest dividends. Every convention you document is a convention the agent follows automatically instead of you catching it in review.

Architecture Patterns: Prevents the agent from fighting your architecture. If you use CQRS and the agent writes a handler that directly calls the database, you've wasted time. Document the pattern, and the agent follows it.

What NOT to Do: Arguably the most important section. Agents are biased toward the most common patterns in their training data. If the most common .NET pattern is to use AutoMapper but you don't, you need to say so explicitly. Negative constraints are as valuable as positive ones.

Path-Scoped Rules

Your main context file handles project-wide conventions. Path-scoped rules handle the specifics of different areas of your codebase.

In Claude Code, these go in `.claude/rules/` with glob patterns. In Cursor, they go in `.cursor/rules/`. The concept is the same: rules that only apply when the agent is working with matching files.

Example: Controller Rules

File: `.claude/rules/controllers.md`

Applies to: `src/Api/Controllers/**`

```
# Controller Conventions

Controllers are thin. They do three things:
1. Validate the request (model binding + FluentValidation)
2. Send a command/query via MediatR
3. Map the result to an HTTP response

No business logic in controllers. No direct database access.
No try/catch (the global exception middleware handles errors).

## Template
```

[ApiController]

[Route("api/v1/[controller]")]

[Authorize]

public class OrdersController(ISender sender) : ControllerBase

{

```
[HttpGet("{id:guid}")]

public async Task> Get(Guid id)

{

var result = await sender.Send(new GetOrderQuery(id));

return result.Match>(

success => Ok(success),

notFound => NotFound()

);

}

}
```

Example: Test Rules

File: `.claude/rules/tests.md`

Applies to: `tests/**`

```
# Testing Conventions

Use xUnit, FluentAssertions, NSubstitute. No other test libraries.

## Unit Tests
- Test one behavior per test method
- Use Arrange/Act/Assert structure (with blank line separators)
- Use NSubstitute for mocking interfaces, never concrete classes
- Test name format: MethodName_Scenario_ExpectedResult

## Integration Tests
- Use WebApplicationFactory<Program>
- Each test class creates its own test database
- Seed data using builder pattern (see tests/Builders/)
- Always dispose/clean up database after test class

## Example
```

public class CreateOrderHandlerTests

{

private readonly IOrderRepository _repository = Substitute.For();

private readonly CreateOrderHandler _handler;

public CreateOrderHandlerTests()

{

_handler = new CreateOrderHandler(_repository);

}

[Fact]

public async Task Handle_ValidOrder_ReturnsCreatedResult()

{

```
// Arrange

var command = new CreateOrderCommand("SKU-001", 2);

// Act

var result = await _handler.Handle(command, CancellationToken.None);

// Assert

result.IsSuccess.Should().BeTrue();

await _repository.Received(1).AddAsync(Arg.Any());

}

}
```

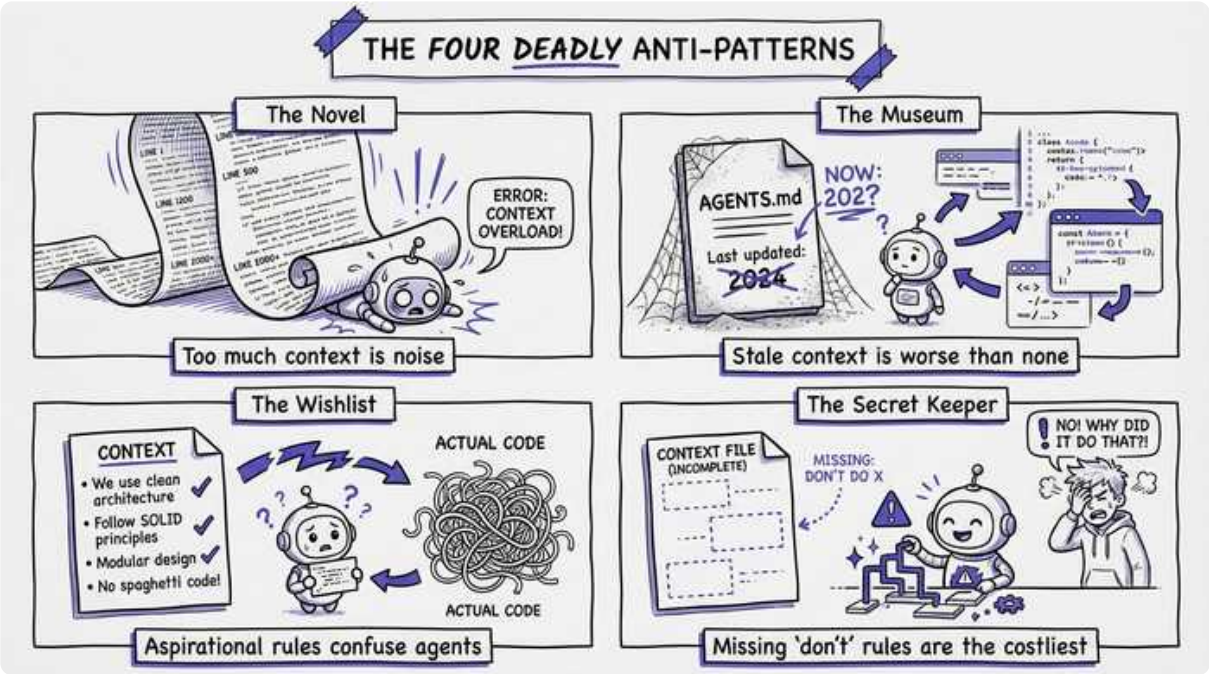
When to Add Path-Scoped Rules

Don't create them all upfront. Add them reactively:

1. Agent writes code in a specific area that doesn't follow your conventions
2. You correct it during review
3. You add a rule so it doesn't happen again

This is the "build context files gradually" principle that experts recommend. Start with the main AGENTS.md. Add scoped rules as patterns emerge. Over a few weeks, your context layer becomes comprehensive.

Context File Anti-Patterns



The Four Anti-Patterns

I've seen developers torpedo their own context engineering in a few predictable ways.

The Novel

A 2,000-line context file that documents every class, every method, every database table. This is counterproductive. Models degrade with too much context. Keep your context file focused on conventions and decisions, not documentation. If the agent needs to understand a specific class, it can read the source code.

Rule of thumb: Your AGENTS.md should be under 200 lines. If it's longer, you're documenting your codebase, not your conventions.

The Museum

A context file that was written once and never updated. Your project evolves. Your conventions evolve. When you add a new pattern, update the context file. When you change a convention, update the context file. A stale context file produces stale code.

Practice: Review your context file every two weeks. Does it still reflect how you actually work? Update what's changed. Remove what's obsolete.

The Wishlist

A context file full of aspirational conventions that the codebase doesn't actually follow. "All methods should be under 20 lines" is useless if your codebase is full of 100-line methods. The agent will either follow the rule (and produce code that's inconsistent with the existing code) or follow the existing code (and ignore your rule).

Rule: Your context file should document what IS, not what you wish it were. If you want to change a convention, change the code first, then update the file.

The Secret Keeper

A context file that omits the negative constraints. What the agent should NOT do is often more important than what it should do. If you've made an architectural decision to avoid a popular pattern (no AutoMapper, no repository base class, no generic controllers), the agent needs to know. Otherwise it will gravitate toward the popular pattern because that's what the training data suggests.

Making Your Codebase AI-Readable

Your context file is only part of the equation. Your codebase itself is context, and how you structure it dramatically affects agent output quality.

Clear Naming

Agents don't have the team knowledge that tells you "OrderProcessor" actually handles payments. They take names at face value. If your class names, method names, and variable names clearly describe what they do, agents produce better code.

This was always a best practice. With agents, it's a force multiplier.

Small, Focused Files

Agents work better with small files that do one thing. A 2,000-line God class is hard for an agent to modify correctly because it can't hold the entire mental model. A 50-line handler is trivial for an agent to understand and modify.

Again, this was always good engineering. Agents just make the payoff more obvious.

Documented Decisions

When you make a non-obvious architectural decision, document it. Not in a wiki that nobody reads, but in a comment or an ADR (Architecture Decision Record) in the repo. Agents read comments. They read markdown files in the repo. They don't read your Confluence.

```
// DECISION: We use raw SQL for this query instead of LINQ because
// EF Core generates a suboptimal execution plan for this specific
// join pattern. See ADR-007 for benchmarks.
```

That comment saves the agent from "helpfully" converting your raw SQL to LINQ.

Comprehensive Test Suite

Your test suite is the most powerful form of context you have. It tells the agent:

- What the code is supposed to do (specification)
- What edge cases matter (boundary conditions)
- What patterns to follow (test structure and conventions)
- Whether its changes work (automated verification)

A codebase with 80%+ test coverage is dramatically easier for agents to work with than one with 10% coverage. The full book includes a chapter on testing strategy for agentic development, but the short version is: invest in tests now. They pay for themselves many times over in the agent era.

The Context Engineering Workflow

Here's the practical workflow for building and maintaining your context layer:

Week 1: The Foundation

1. Create AGENTS.md with the template above, adapted to your project
2. Fill in every section honestly (what IS, not what you wish)
3. Start your next coding task using an agent with this context file
4. Note where the agent still gets it wrong

Week 2: The Refinement

1. Add path-scoped rules for the areas where the agent struggled
2. Add "What NOT to Do" entries for every convention the agent violated
3. Review and tighten your AGENTS.md based on the first week's experience
4. Start writing test-first for agent tasks (preview of Chapter 5)

Week 3: The Expansion

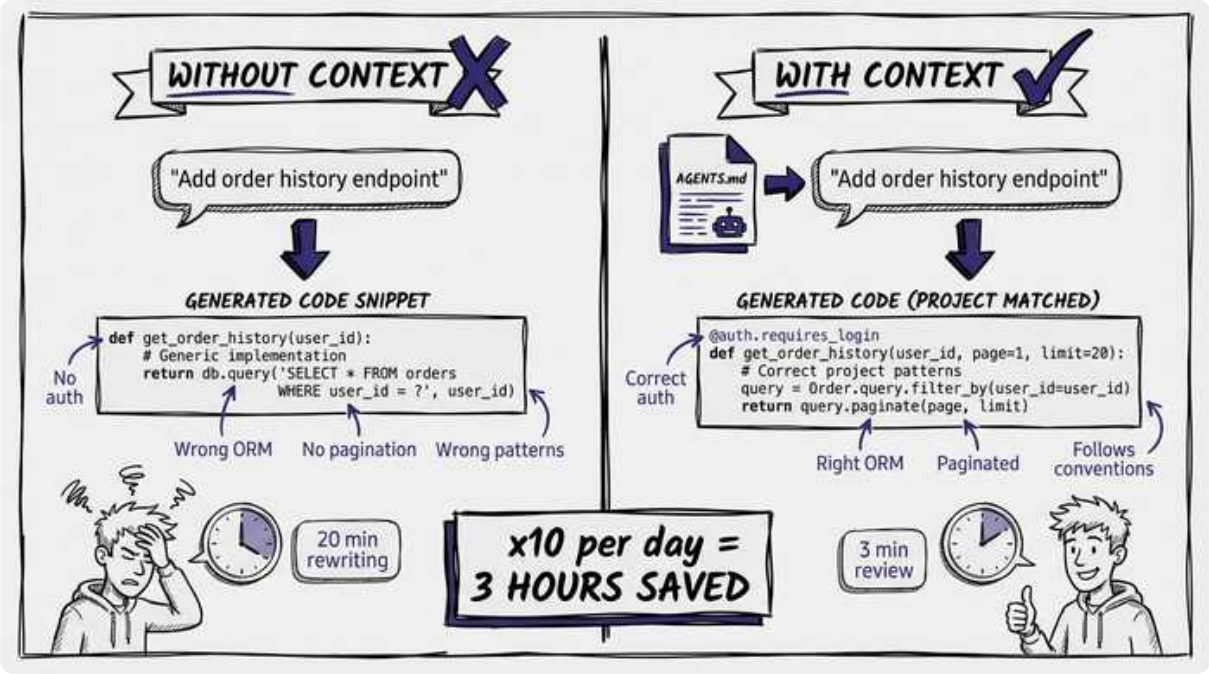
1. Add rules for remaining code areas (frontend, infrastructure, etc.)
2. Document architectural decisions that agents keep getting wrong
3. Add examples (like the controller template above) for complex patterns
4. Share the context files with your team for feedback

Week 4: The Habit

1. Context file updates become part of your PR process
2. When you change a convention, update the context file in the same PR
3. Review the full context layer: is anything stale? Redundant? Missing?
4. Measure: are agent outputs getting better over time?

By the end of this process, you'll have a context layer that makes agent interactions productive by default. New team members (human or AI) can read your context files and immediately produce code that fits.

A Real Example



Without Context vs. With Context

Here's a condensed example of how context engineering transforms agent output. Same task, same agent, different context.

Without context file:

Prompt: "Add an endpoint to get a user's order history"

Agent produces: a controller with direct DbContext access, AutoMapper for mapping, synchronous calls, no authentication, returns a List instead of paginated results.

You spend 20 minutes rewriting it to fit your architecture.

With context file:

Same prompt: "Add an endpoint to get a user's order history"

Agent produces: a controller that sends a GetOrderHistoryQuery via MediatR, a handler that uses the repository pattern, manual mapping, async throughout, [Authorize] attribute, paginated response with proper HTTP headers.

You spend 3 minutes reviewing and approve it.

The difference is 17 minutes on a single task. Multiply that by every agent interaction across a day, a week, a month. Context engineering doesn't just help. It compounds.

Key Takeaways

1. **Create an AGENTS.md now.** Use the template. Adapt it. It takes 30 minutes and pays off immediately.
1. **Document conventions, not code.** Your context file tells the agent how to work, not what the code does.
1. **Negative constraints are critical.** What NOT to do is as important as what to do.
1. **Build context gradually.** Start with the main file. Add scoped rules as patterns emerge.
1. **Your codebase is context.** Clean naming, small files, documented decisions, and comprehensive tests make everything better.
1. **Maintain it.** A stale context file is worse than no context file because it produces confidently wrong output.

Context engineering is the skill that separates developers who are slower with AI from developers who are dramatically faster. Invest in it. Everything else in this book builds on this foundation.

CHAPTER 5

The TDD Agent Loop

(Chapter 13 in the full book)

This is Chapter 13 in the full book.

This is the workflow that changed everything for me. Not gradually, not theoretically, but immediately and measurably. The TDD Agent Loop is the single highest-confidence way to work with AI coding agents, and once you've used it, going back feels reckless.

The concept is simple: write the tests first, then let the agent write the code to make them pass.

That's it. That's the workflow. But the devil (and the value) is in the details of how you structure the tests, how you prompt the agent, how you handle failures, and how you review the result.

Why TDD + Agents Is a Perfect Match

Test-driven development has been around since the early 2000s. It's always been a good practice. But it had a cost: writing tests first is slower than writing code first, at least in the short term. You're essentially writing the code twice, once as a specification (the tests) and once as an implementation.

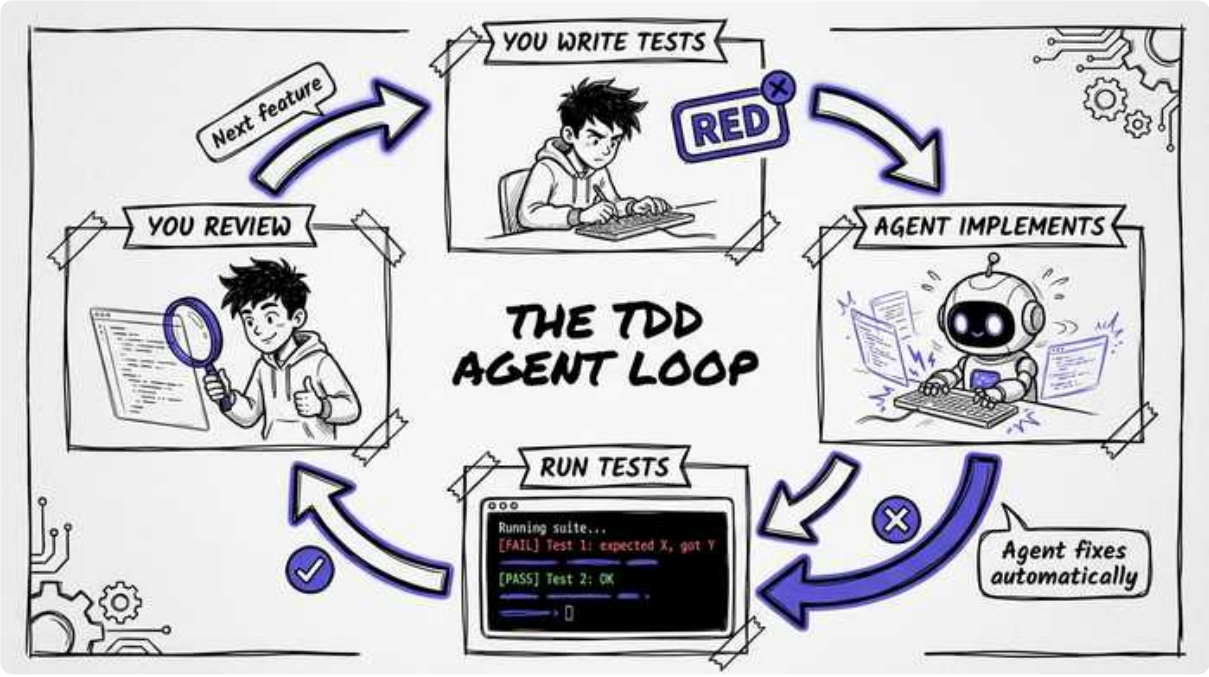
Agents eliminate the second half of that cost. You write the tests. The agent writes the implementation. The total time is less than writing just the implementation by hand, because:

1. **The tests serve as an unambiguous spec.** The agent doesn't have to guess what you want. The tests define it.

2. **The agent can self-correct.** When a test fails, the agent sees the failure message, understands what went wrong, and fixes it. No human in the loop needed for mechanical corrections.
3. **Your review is simplified.** Instead of reviewing the code to see if it works, you verify that the tests are comprehensive and the code is clean. The "does it work?" question is already answered.
4. **Confidence is built-in.** When all tests pass, you know the code does what you specified. Not what the agent assumed. What you specified.

Addy Osmani put it well: "The more comprehensive your tests, the more confidently you can delegate." The TDD Agent Loop takes this to its logical conclusion: your tests ARE your delegation.

The Loop in Detail



The TDD Agent Loop

Here's the workflow, step by step.

Step 1: Understand the Requirement

Before writing anything, be clear on what you're building. This sounds obvious, but agents amplify confusion. If you're unclear about the requirement, you'll write vague tests, the agent will write code that passes vague tests, and you'll end up with code that technically works but doesn't actually solve the problem.

Spend 5 minutes with the requirement. Write it in plain language if it helps:

```
I need a service that calculates shipping costs.
- Free shipping for orders over $100
- Flat rate $5.99 for standard shipping
- Flat rate $12.99 for express shipping
- No shipping to PO boxes for express
- Tax is NOT applied to shipping costs
```

This isn't a prompt for the agent yet. This is your own understanding.

Step 2: Write the Tests

Now write the tests. All of them. Before any implementation exists.

```
public class ShippingCalculatorTests
{
    private readonly ShippingCalculator _calculator = new();

    [Fact]
    public async Task Calculate_OrderOver100_StandardShipping_ReturnsFree()
    {
        // Arrange
        var order = new Order { Total = 150.00m, ShippingMethod =
ShippingMethod.Standard };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(0m);
    }

    [Fact]
    public async Task
Calculate_OrderUnder100_StandardShipping_ReturnsFlatRate()
    {
        // Arrange
        var order = new Order { Total = 50.00m, ShippingMethod =
ShippingMethod.Standard };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(5.99m);
    }

    [Fact]
    public async Task Calculate_OrderOver100_ExpressShipping_ReturnsFree()
    {
```

```
        // Arrange
        var order = new Order { Total = 200.00m, ShippingMethod =
ShippingMethod.Express };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(0m);
    }

    [Fact]
    public async Task Calculate_OrderUnder100_ExpressShipping_ReturnsFlatRate()
    {
        // Arrange
        var order = new Order { Total = 75.00m, ShippingMethod =
ShippingMethod.Express };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(12.99m);
    }

    [Fact]
    public async Task
Calculate_ExpressToPoBox_ThrowsInvalidOperationException()
    {
        // Arrange
        var order = new Order
        {
            Total = 50.00m,
            ShippingMethod = ShippingMethod.Express,
            Address = new Address { Line1 = "PO Box 123" }
        };
```

```
        // Act
        var act = () => _calculator.CalculateAsync(order);

        // Assert
        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage("*PO Box*");
    }

    [Fact]
    public async Task Calculate_ShippingCost_DoesNotIncludeTax()
    {
        // Arrange
        var order = new Order
        {
            Total = 50.00m,
            ShippingMethod = ShippingMethod.Standard,
            TaxRate = 0.08m
        };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(5.99m);
        result.TaxOnShipping.Should().Be(0m);
    }

    [Theory]
    [InlineData(99.99, 5.99)]
    [InlineData(100.00, 5.99)]
    [InlineData(100.01, 0)]
    public async Task Calculate_BoundaryValues_ReturnsCorrectCost(
        decimal orderTotal, decimal expectedShipping)
    {
        // Arrange
```

```
        var order = new Order { Total = orderTotal, ShippingMethod =
ShippingMethod.Standard };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(expectedShipping);
    }
}
```

A few things to notice about these tests:

They're thorough. Happy path, edge cases, boundary values, error conditions. This isn't minimal test coverage. This is a complete specification.

They define the API. The tests force you to decide on the interface: `ShippingCalculator` with a `CalculateAsync` method that takes an `Order` and returns a result with `ShippingCost` and `TaxOnShipping`. You're designing the API, not the agent.

They won't compile yet. `ShippingCalculator`, `Order`, `ShippingMethod`, `Address`, none of these exist. That's fine. That's the point. The tests define what needs to exist.

The boundary test is critical. Is $100.00 exactly free or not? You decided: it's NOT free (the tests show $100.00 maps to $5.99, and $100.01 maps to free). That's a business decision you made explicitly. Without the test, the agent would guess, and guessing business rules is how bugs are born.

> ***Note:*** *These examples use C# with xUnit and FluentAssertions, but the TDD Agent Loop works identically with any language and test framework. Python with pytest, TypeScript with Jest, Go with the testing package, Java with JUnit. The pattern is universal.*

Step 3: Prompt the Agent

Now you hand it to the agent. The prompt is simple because the tests do the heavy lifting:

```
Make all the tests in ShippingCalculatorTests pass.

The tests are in tests/Domain.Tests/ShippingCalculatorTests.cs.
Create the necessary types and implementation in src/Domain/.

Run the tests after implementation to verify they pass.
```

That's it. No 50-line prompt. No detailed specification. The tests ARE the specification.

If you've set up your AGENTS.md (Chapter 4), the agent already knows your project structure, coding conventions, and architecture patterns. If you have path-scoped rules, it knows the conventions for the Domain project specifically.

Step 4: Let the Agent Iterate

Here's where the magic happens. The agent will:

1. Read the test file
2. Understand what types need to exist (`Order` , `Address` , `ShippingMethod` , `ShippingResult` , `ShippingCalculator`)
3. Create those types
4. Implement `CalculateAsync`
5. Run `dotnet test`
6. See which tests fail (if any)
7. Fix the failures
8. Run the tests again
9. Repeat until all tests pass

This iteration loop is the key to the whole workflow. The agent isn't generating code and hoping it works. It's generating code, testing it, and fixing what's broken. The feedback loop is tight, automatic, and objective.

In my experience, most tasks complete in 1-3 iterations. The agent gets it mostly right on the first try (thanks to the tests being clear and the context file providing conventions), catches a bug or two on the test run, and fixes them.

Step 5: Review the Result

When all tests pass, the agent presents you with the implementation. Now you review.

But here's what's different about this review compared to reviewing code without the TDD loop: **you already know it works.** The tests pass. The behavior is correct. Your review focuses on:

Code quality: Is it clean? Is it readable? Does it follow your conventions?

Design decisions: Did the agent make reasonable choices about how to structure the code? Is the approach maintainable?

Edge cases you missed: Did the agent handle any cases your tests didn't cover? (Sometimes agents add defensive code that's valuable. Sometimes they add unnecessary complexity.)

Security and performance: Does anything look risky? For a shipping calculator, probably not. For an authentication handler, this review is critical.

Here's what the agent might produce:

```
namespace Domain;

public enum ShippingMethod
{
    Standard,
    Express
}

public record Address
{
    public required string Line1 { get; init; }
    public string? Line2 { get; init; }
    public string? City { get; init; }
    public string? State { get; init; }
    public string? ZipCode { get; init; }
}

public record Order
{
    public decimal Total { get; init; }
    public ShippingMethod ShippingMethod { get; init; }
    public Address? Address { get; init; }
    public decimal TaxRate { get; init; }
}

public record ShippingResult
{
    public decimal ShippingCost { get; init; }
    public decimal TaxOnShipping { get; init; }
}

public class ShippingCalculator
{
    private const decimal FreeShippingThreshold = 100.01m;
    private const decimal StandardRate = 5.99m;
    private const decimal ExpressRate = 12.99m;
```

```
    public Task<ShippingResult> CalculateAsync(Order order)
    {
        if (order.ShippingMethod == ShippingMethod.Express &&
            order.Address?.Line1?.Contains("PO Box",
StringComparison.OrdinalIgnoreCase) == true)
        {
            throw new InvalidOperationException(
                "Express shipping is not available for PO Box addresses.");
        }

        var isFreeShipping = order.Total >= FreeShippingThreshold;

        var shippingCost = isFreeShipping
            ? 0m
            : order.ShippingMethod == ShippingMethod.Express
                ? ExpressRate
                : StandardRate;

        return Task.FromResult(new ShippingResult
        {
            ShippingCost = shippingCost,
            TaxOnShipping = 0m
        });
    }
}
```

Review checklist:

- ✅ All tests pass
- ✅ Uses records for DTOs (matches our convention)
- ✅ Constants extracted (clean)
- ✅ Async pattern correct (even though this is synchronous internally, the interface is async for future flexibility)
- ✅ PO Box check is case-insensitive (good defensive choice)

- ⚠️ The "PO Box" detection is naive (only checks Line1, only checks exact string). Might want to improve this later.

That last point is a judgment call. The agent implemented what the tests specified. If you want better PO Box detection, write a test for it. That's the loop.

Step 6: Expand (Optional)

Sometimes after the initial implementation, you think of additional cases:

```
[Fact]
public async Task Calculate_ExpressToPoBox_CaseInsensitive_ThrowsException()
{
    var order = new Order
    {
        Total = 50.00m,
        ShippingMethod = ShippingMethod.Express,
        Address = new Address { Line1 = "po box 456" }
    };

    var act = () => _calculator.CalculateAsync(order);

    await act.Should().ThrowAsync<InvalidOperationException>();
}

[Fact]
public async Task Calculate_ExpressToPoBoxInLine2_ThrowsException()
{
    var order = new Order
    {
        Total = 50.00m,
        ShippingMethod = ShippingMethod.Express,
        Address = new Address { Line1 = "123 Main St", Line2 = "PO Box 789" }
    };

    var act = () => _calculator.CalculateAsync(order);

    await act.Should().ThrowAsync<InvalidOperationException>();
}
```

Run the agent again: "Make the new tests pass without breaking existing tests." The agent updates the implementation. You review. Cycle complete.

When the Loop Shines

The TDD Agent Loop is particularly effective for:

Business Logic

Services, calculators, validators, anything with clear inputs and outputs. These are the tasks where tests are easiest to write and agent implementations are most reliable.

API Endpoints (Integration Tests)

Write integration tests that hit your endpoints through `WebApplicationFactory`, specifying the expected request/response contracts. Let the agent build the full vertical slice: controller, handler, repository calls.

```
[Fact]
public async Task CreateOrder_ValidRequest_Returns201WithLocation()
{
    // Arrange
    var client = _factory.CreateClient();
    var request = new CreateOrderRequest("SKU-001", 2);

    // Act
    var response = await client.PostAsJsonAsync("/api/v1/orders", request);

    // Assert
    response.StatusCode.Should().Be(HttpStatusCode.Created);
    response.Headers.Location.Should().NotBeNull();

    var order = await response.Content.ReadFromJsonAsync<OrderResponse>();
    order.Should().NotBeNull();
    order!.Sku.Should().Be("SKU-001");
    order.Quantity.Should().Be(2);
}
```

This single test tells the agent to create the endpoint, the request/response types, the handler, and wire everything up. The agent builds the full feature to make the test pass.

Refactoring

Write tests that capture the current behavior, then ask the agent to refactor the implementation. The tests ensure the refactoring doesn't break anything. This is one of the safest ways to let agents work on existing code.

Bug Fixes

Write a test that reproduces the bug (it should fail). Ask the agent to make it pass without breaking existing tests. The bug fix is verified by definition.

When the Loop Struggles

The TDD Agent Loop isn't a silver bullet. It works poorly for:

UI work. Testing visual output is hard to express in unit/integration tests. You can test behavior (click this, expect this state) but visual correctness requires human eyes.

Exploratory work. When you don't know what you want yet, you can't write tests for it. Use agents in chat/explore mode first, figure out the approach, then switch to TDD for the implementation.

Infrastructure and configuration. Testing Terraform modules, Docker configurations, or CI/CD pipelines requires different tools than `dotnet test`. The loop concept still applies (define expected behavior, let the agent implement, verify) but the test framework is different.

Performance optimization. Tests verify correctness, not performance. An agent can make all tests pass with an $O(n^2)$ algorithm when you needed $O(n \log n)$. Performance review remains a human judgment.

The Full Cycle in Practice

Let me walk through a realistic workday scenario using the TDD loop.

9:00 AM: You pick up a ticket to add a discount code system. You spend 15 minutes understanding the requirements and sketching the domain model on paper.

9:15 AM: You write 8 tests covering: valid discount application, expired code rejection, usage limit enforcement, minimum order threshold, percentage vs. fixed discounts, stacking rules, and case-insensitive code lookup.

9:45 AM: You prompt the agent: "Make all tests in DiscountServiceTests pass. Create types in src/Domain/Discounts/. Follow the existing patterns in src/Domain/Shipping/ for reference."

9:50 AM: Agent completes. All 8 tests pass on the second iteration (it missed the case-insensitive lookup on the first try, caught it when the test failed, fixed it).

9:55 AM: You review the implementation. Clean code, follows conventions, handles all cases. You notice it created a separate `DiscountCode` entity and `IDiscountRepository` interface, which fits your architecture. You approve.

10:00 AM: You write 3 integration tests for the API endpoint: apply discount to order (happy path), apply invalid code (400 response), and apply to order below minimum (422 response).

10:10 AM: Agent prompt: "Make the new integration tests pass. Create the endpoint, handler, and wire up the discount service."

10:15 AM: Agent completes. All tests pass. You review the controller, handler, and DI registration. Everything follows your conventions (thanks to the AGENTS.md file). You approve and commit.

Total time: 1 hour 15 minutes for a complete discount code feature with full test coverage, clean architecture, and verified behavior.

Without the TDD loop, this would be 3-4 hours of implementation, manual testing, debugging, and review. With naive agent use (no tests, no context), it might be 2-3

hours plus accumulated technical debt from code that "works" but doesn't fit your architecture.

Making It Stick

The hardest part of the TDD Agent Loop isn't the technique. It's the habit change. You have to resist the urge to prompt the agent immediately and instead invest the 15-30 minutes in writing tests first.

Here's what helps:

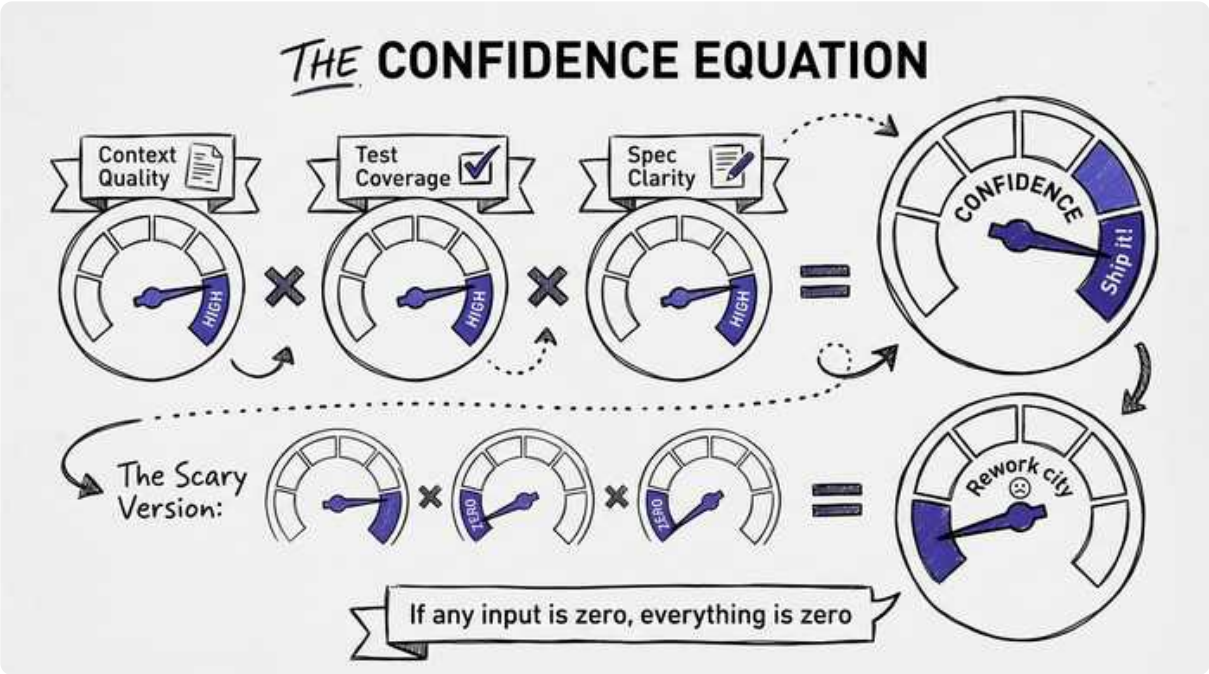
Start small. Pick one task tomorrow and try the loop. A bug fix is ideal, because writing a reproducing test is natural. See how it feels. See the results.

Timebox the tests. Give yourself 30 minutes maximum to write tests. If the tests are taking longer than that, the task might be too big. Break it down.

Keep a "tests I wish I'd written" list. When you skip the TDD loop and regret it (because the agent produced garbage you had to rewrite), note it. The list builds the habit by making the cost of skipping visible.

Pair the loop with context files. The TDD Agent Loop is good on its own. Combined with a well-maintained AGENTS.md, it's transformative. The context file handles the "how" (conventions, patterns, structure), and the tests handle the "what" (behavior, correctness, edge cases). Together, they constrain the agent into a small space where almost any output is acceptable.

The Confidence Equation



The Confidence Equation

Here's the mental model that ties this chapter to the rest of the book:

Agent confidence = Context quality × Test coverage × Specification clarity

If any of those three is zero, your confidence in the output is zero, regardless of how good the other two are. But when all three are high:

- **Context** tells the agent how to write (conventions, patterns, architecture)
- **Tests** tell the agent what to write (behavior, edge cases, contracts)
- **Specification** tells the agent why (business rules, constraints, intent)

The TDD Agent Loop maximizes the first two. The specification (your test names, your requirement notes, your prompt) handles the third. When all three are dialed in, you can trust agent output at a level that makes the productivity gains real, not illusory.

This is the answer to the METR paradox. The developers in that study had low context, no tests-first workflow, and vague specifications. Of course they were slower. The agents had nothing to work with except the prompt and the code.

Give agents context, tests, and clarity, and the equation inverts. You're not 19% slower. You're 2-3x faster. And you can prove it, because the tests prove it.

That's the TDD Agent Loop. Write the tests. Let the agent handle the rest. Review with confidence.

You've just read 5 of 29 chapters. Here's everything else waiting for you.

Part I: The Shift (Chapters 4-5)

Chapter 4: The Agentic Engineering Mindset — The mental models that separate productive agentic developers from frustrated ones. How to think about agents as junior developers with perfect memory and zero judgment.

Chapter 5: From Vibe Coding to Vibe Engineering — Karpathy's evolution from casual prompting to serious discipline, and what that trajectory teaches us about where the field is headed.

Part II: Context Engineering (Chapters 7-10)

Chapter 7: Path-Scoped Rules and When to Use Them — Beyond the main context file. How to create modular, targeted rules for different parts of your codebase.

Chapter 8: Skills: Teaching Your Agent New Tricks — The skills system in Claude Code and how to create reusable, LLM-invokable instruction sets for complex workflows.

Chapter 9: MCP Servers and External Tool Integration — Connecting agents to databases, APIs, and custom tooling through the Model Context Protocol. Includes building an MCP server in C#.

Chapter 10: Designing an AI-Friendly Codebase — Architecture and code organization patterns that make agents dramatically more effective. The practices that were always good engineering but now have 10x payoff.

Part III: The Tools (Chapters 11-12)

Chapter 11: IDE Agents: Copilot, Cursor, and Windsurf — When to use each, how to configure them, and the specific workflows where each excels.

Chapter 12: CLI Agents: Claude Code and Codex CLI — Terminal-native agent workflows for power users. Sub-agents, parallel execution, and scripting agent tasks.

Part IV: The Workflows (Chapters 14-17)

Chapter 14: Spec-First Development — Write the design doc, break it into tasks, let agents implement. The workflow Simon Willison uses daily.

Chapter 15: Error-Message-Driven Debugging — Paste the error, get the fix. How to structure error context so agents diagnose accurately.

Chapter 16: Codebase Archaeology and Documentation — Using agents to understand and document existing code. Generating internal docs, architecture diagrams, and onboarding guides.

Chapter 17: Migration Generation — Framework upgrades, database migrations, API versioning. The tasks agents handle exceptionally well with the right context.

Part V: Scaling Up (Chapters 18-22)

Chapter 18: The Parallel Agent Lifestyle — Running 3-5 agents simultaneously on different tasks. Git worktrees, task decomposition, and review orchestration.

Chapter 19: Background Agents and Async Workflows — Fire-and-forget from your phone. GitHub Copilot Coding Agent, Codex Cloud, and overnight automation.

Chapter 20: Multi-Agent Coordination — Agent teams working the same codebase. File-based locking, orchestration patterns, and conflict resolution.

Chapter 21: Agent + CI/CD Integration — Agents that react to failing builds, fix issues automatically, and manage deployment pipelines.

Chapter 22: Cost Management and Token Economics — Nobody talks about this. The real cost of running multiple agents, and how to optimize spend without sacrificing productivity.

Part VI: .NET-Specific Patterns (Chapters 23-26)

Chapter 23: Context Files for .NET Projects — Templates for ASP.NET Core, Blazor, MAUI, and class library projects. The .NET-specific conventions agents need to know.

Chapter 24: Agents and .NET Aspire — Using the Aspire dashboard for agent telemetry, building cloud-native AI services, and the Aspire + agent development workflow.

Chapter 25: EF Core Migrations with Agents — Database schema evolution through agents. Migration generation, seed data, and schema review workflows.

Chapter 26: Building Multi-Agent Apps with Microsoft Agent Framework — Building your own multi-agent systems in .NET. The Agent Framework, Semantic Kernel integration, and production deployment.

Part VII: The Discipline (Chapters 27-29)

Chapter 27: Security Review with Agents — OWASP Top 10 scanning, dependency auditing, secrets detection, and systematic security review workflows.

Chapter 28: When NOT to Use Agents — The tasks where agents hurt. How to recognize them, and the discipline to do the work by hand when that's the right call.

Chapter 29: The 30-Day Transformation Plan — Day-by-day plan to go from wherever you are today to a fully operational agentic workflow. Concrete actions, milestones, and checkpoints.

Get the Full Book

The full version of **The Agentic Developer** includes all 29 chapters, downloadable context file templates, a companion GitHub repository with working code examples, and access to updates as the field evolves.

Get it at: dotnetacademy.io/agentic-developer

Thanks for reading. Now go write a context file and break the METR paradox.

Ready for the Full Book?

The complete version includes all 29 chapters, downloadable context file templates, companion GitHub repository with working code examples, and access to updates as the field evolves.

The Agentic Developer is available at your next step in mastery.

Thanks for reading this free playbook.

Now go write a context file and break the METR paradox.